

Lesson 5 — Evaluating Classification Models

In Lesson 4, we learned:

```
Classification
  ↓
predict categories
  ↓
0 / 1
Spam / Not Spam
Fraud / Not Fraud
```

Now the important question is:

How do we know whether our classification model is actually good?

We'll learn:

1. Accuracy
2. Confusion Matrix
3. TP, TN, FP, FN
4. Precision
5. Recall
6. F1 Score
7. When to use which metric

1. Accuracy

We already saw accuracy in Lesson 4.

Suppose we have 100 predictions:

```
Correct predictions = 90
Wrong predictions  = 10
```

Then:

```
Accuracy = 90 / 100
          = 0.90
          = 90%
```

In Scikit-learn:

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
```

Simple enough.

But accuracy can sometimes be misleading.

That's why we need the confusion matrix.

2. Confusion Matrix

Suppose we're building a fraud detection system.

Our classes are:

```
0 → Not Fraud
1 → Fraud
```

The model makes predictions.

We compare:

```
Actual value
↓
Predicted value
```

The confusion matrix organizes those results.

		Predicted	
		Not Fraud	Fraud
Actual Not Fraud		TN	FP
Actual Fraud		FN	TP

These four terms are extremely important in ML.

3. TP — True Positive

True Positive

The model predicted positive, and it was actually positive.

Fraud example:

```
Actual:   Fraud
Predicted: Fraud
```

✔ Correct.

Therefore:

```
TP = True Positive
```

4. TN — True Negative

True Negative

The model predicted negative, and it was actually negative.

```
Actual:   Not Fraud
Predicted: Not Fraud
```

✔ Correct.

Therefore:

TN = True Negative

5. FP — False Positive

False Positive

The model predicted positive, but it was actually negative.

Actual: Not Fraud
Predicted: Fraud

✗ Wrong.

This is called a false alarm.

For example:

Normal transaction
↓
Model says:
"FRAUD!"
↓
But it wasn't fraud

That's:

FP = False Positive

6. FN — False Negative

False Negative

The model predicted negative, but it was actually positive.

Actual: Fraud
Predicted: Not Fraud

✗ Wrong.

This can be very dangerous.

Fraud transaction
↓
Model says:
"Not Fraud"
↓
Fraud gets through

Therefore:

FN = False Negative

7. Remember TP/TN/FP/FN Like This

The easiest way:

		MODEL	
		Positive	Negative
ACTUAL	Positive	TP	FN
ACTUAL	Negative	FP	TN

The first letter tells you whether the prediction was:

T = True
F = False

The second tells you what the model predicted:

P = Positive
N = Negative

So:

TP → True + Positive
TN → True + Negative
FP → False + Positive
FN → False + Negative

8. Example

Imagine 100 transactions.

The model produces:

TP = 30
TN = 60
FP = 5
FN = 5

So:

		Predicted	
		Not Fraud	Fraud
Actual	Not Fraud	60	5
Actual	Fraud	5	30

Total:

$60 + 5 + 5 + 30 = 100$

Correct predictions:

TN + TP
= 60 + 30
= 90

Therefore:

Accuracy = $90 / 100$
= 90%

9. Confusion Matrix in Scikit-learn

Import:

```
from sklearn.metrics import confusion_matrix
```

Then:

```
cm = confusion_matrix(y_test, predictions)

print(cm)
```

You might get:

```
[[60  5]
 [ 5 30]]
```

Interpretation:

```
[[TN FP]
 [FN TP]]
```

So:

```
TN = 60
FP = 5
FN = 5
TP = 30
```

10. Visualizing It

You can also use:

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(
    y_test,
    predictions
)
```

This gives you a visual confusion matrix.

11. Precision

Now we get to one of the most important metrics.

Precision asks:

Of everything the model predicted as positive, how many were actually positive?

Formula:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Using our example:

```
TP = 30
```

FP = 5

Therefore:

$$\begin{aligned}\text{Precision} &= 30 / (30 + 5) \\ &= 30 / 35 \\ &\approx 0.857\end{aligned}$$

So:

Precision $\approx$ 85.7%

12. Think About Precision Like This

Suppose your model says:

"This transaction is FRAUD"

100 times.

You want to know:

How many of those 100 were actually fraud?

That's precision.

High precision means:

When model says FRAUD
↓
Usually actually FRAUD

So precision is particularly important when false positives are costly.

13. Recall

Recall asks a different question:

Of all the actual positive cases, how many did the model successfully find?

Formula:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Our example:

TP = 30
FN = 5

Therefore:

$$\begin{aligned}\text{Recall} &= 30 / (30 + 5) \\ &= 30 / 35 \\ &\approx 85.7\%\end{aligned}$$

14. Precision vs Recall

This is one of the most important concepts to understand.

Precision

Of predicted positives,
how many were actually positive?

Recall

Of actual positives,
how many did we find?

Think:

PRECISION
"Can I trust my positive predictions?"

RECALL
"Did I find most of the actual positives?"

15. Fraud Detection Example

Imagine there are:

100 actual fraud transactions

Your model catches:

95

but misses:

5

Then recall is:

$95 / (95 + 5)$
= 95%

That's excellent recall.

Why?

Because we caught 95% of actual fraud.

16. Medical Diagnosis Example

Suppose we're detecting a serious disease.

Actual sick people
↓
100

Model detects:

98

and misses:

2

High recall is extremely important because we don't want sick patients to be incorrectly classified as healthy.

Those missed cases are:

FN

False negatives.

17. Precision vs Recall — Simple Memory Trick

Remember:

Precision

P = Predicted Positive

Focuses on:

Predicted positives

Recall

Think:

Recall = Retrieve actual positives

Focuses on:

Actual positives

18. F1 Score

Sometimes we want a balance between:

Precision
+
Recall

That's where F1 Score comes in.

Formula:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

You don't need to memorize the mathematical formula yet.

Conceptually:

```
Precision
+
Recall
↓
F1 Score
```

F1 is high when both precision and recall are high.

19. Scikit-learn Code

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

accuracy = accuracy_score(y_test, predictions)
precision = precision_score(y_test, predictions)
recall = recall_score(y_test, predictions)
f1 = f1_score(y_test, predictions)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

20. Classification Report

Scikit-learn gives us a convenient function:

```
from sklearn.metrics import classification_report

print(classification_report(y_test, predictions))
```

You'll see something similar to:

	precision	recall	f1-score	support
0	0.96	0.97	0.96	70
1	0.96	0.94	0.95	44
accuracy			0.96	114
macro avg	0.96	0.96	0.96	114
weighted avg	0.96	0.96	0.96	114

Don't worry about `macro avg`, `weighted avg`, and `support` yet. We'll come back to them.

For now focus on:

```
precision
recall
f1-score
accuracy
```

21. Complete Lesson 5 Code

Let's put everything together:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Load dataset
data = load_breast_cancer()

X = data.data
y = data.target

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Create model
model = LogisticRegression(max_iter=10000)

# Train
model.fit(X_train, y_train)

# Predict
predictions = model.predict(X_test)

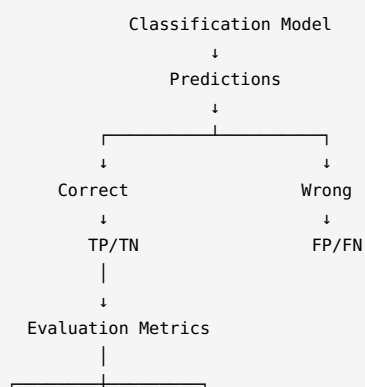
# Accuracy
accuracy = accuracy_score(y_test, predictions)
print("Accuracy:", accuracy)

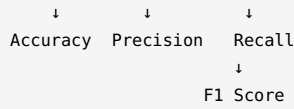
# Confusion Matrix
cm = confusion_matrix(y_test, predictions)
print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, predictions))
```

The Big Picture

You should now understand this:





🔗 When Should You Care About Which Metric?

Situation	Important metric
Balanced/simple problem	Accuracy
False positives are costly	Precision
False negatives are costly	Recall
Want balance between precision & recall	F1
Want detailed error analysis	Confusion Matrix

Example:

Spam detection

Too many legitimate emails marked spam → precision matters.

Disease detection

Missing a sick patient → recall matters.

Fraud detection

Usually both precision and recall matter.

☆ Very Important for Your ML Journey

Don't think:

"My model has 95% accuracy, therefore it's excellent."

Instead ask:

```
95% accuracy
  ↓
How many FP?
How many FN?
  ↓
Precision?
Recall?
F1?
  ↓
Is this model actually useful?
```

That's how you start thinking like an ML practitioner rather than just running Scikit-learn code.

📖 Exercises

Try these yourself:

Q1.

What does TP mean?

Q2.

A transaction is actually fraud, but the model predicts “Not Fraud.”

Is this:

TP / TN / FP / FN

Q3.

A transaction is actually normal, but the model predicts “Fraud.”

Which one?

Q4.

What does precision tell us?

Q5.

What does recall tell us?

Q6.

If missing a positive case is very dangerous, should we care more about precision or recall?

Q7.

What is the purpose of the confusion matrix?

Q8.

Why isn't accuracy always enough?

Next: Lesson 6 — K-Nearest Neighbors (KNN)

We'll learn your first non-linear classification algorithm:

```
New data point
↓
Find nearest points
↓
Look at their classes
↓
Majority vote
↓
Prediction
```